Bachelor of Science in Computer Science Program

2023-2024

LAB Manual

Programming Fundamentals

CSC-101

Department of Information Engineering Technology

National Skills University, Islamabad

Introduction

Course Title: Programming Fundamentals (CSC-101)

Laboratory: Programming Fundamentals Lab

Instructor: Ms. Huma Israr

Designation: Lecturer Information Engineering Technology

Email: huma.israr@nsu.edu.pk

I. Course Objective:

Programming Fundamentals (Lab) CLO's

CLO s	CLO Description	Taxonomy Level	Mapped PLO
CLO-1	**Understand** basic problem solving steps and logic constructs.	C2	PLO-1
CLO-2	**Apply** basic programing concepts.	C3	PLO-2
CLO-3	**Design** and implement algorithms to solve real world problems	C3	PLO-4

Lab CLO's Attainment

CLOs	Level of learning	Mapped OBE PLOs	Teaching Methods	CLO attainment checked in
CLO-1	**Cog-2**	**1**	Lectures, tutorials	Quiz, Mid and Final Paper
CLO-2	**Cog-3**	**2**	Lectures, tutorials	Quiz, Mid and Final Paper
CLO-3	**Cog-3**	**4**	Lectures, tutorials	Quiz, Mid and Final Paper

Lab CLOs Mapping

	Academic Education	Knowledge for Solving Computing Problem	Problem Analysis	Design/Development of Solution	Modern Tool Usage	Individual and Team Work	Communication	Commuting Professionalism and Society	Ethics	Life-long Learning
	PLO1	**PLO2**	**PLO3**	**PLO4**	**PLO5**	**PLO6**	**PLO7**	**PLO8**	**PLO9**	**PLO10**
CLO-1	✓									
CLO-2		✓								
CLO-3				✓						

Program Learning Outcomes	
PLO1: Academic Education	**PLO 6:** Individual and Team work
PLO2: Knowledge of Solving Computing Problem	**PLO 7:** Communication
PLO3: Problem Analysis	**PLO 8:** Computing Professionalism and society
PLO4: Design/ Development of Solution	**PLO 9:** Ethics
PLO5: Modern Tool Usage	**PLO 10:** Lifelong Learning

COURSE OUTCOME

After completing this lab course you will be able to:

1. Understand and build the logic for a given problem. .
2. Write algorithms and Draw a flow chart of a given problem.
3. Recognize and understand the syntax and construction of C++ programming code.
4. Gain experience of procedural language programming.
5. Know the steps involved in compiling, linking and debugging C++ code.
6. Understand using header files.
7. Learn the methods of iteration or looping and branching.
8. Make use of different data-structures like arrays, pointers, structures and files.
9. Understand how to access and use library functions.
10. Understand function declaration and definition.
11. Understand proper use of user defined functions.
12. Write programs to print output on the screen as well as in the files.
13. Apply all the concepts that have been covered in the theory course, and
14. Know the alternative ways of providing solution to a given problem.

II. Grading Policy

Lab Performance (Lab Tasks)/Lab Manual	**25%**
Lab Midterm Exam	**25%**
Lab Final Exam/Open Ended Lab/Semester Lab Project/ Lab Report	**50%**

III. List of Lab Practical's

The objective of the Course is to

1. To introduce students to the basic knowledge of programming fundamentals of C++ language.
2. To impart writing skill of C++ programming to the students and solving problems.
3. To impart the concepts like looping, array, functions, pointers, file, structure.

S.No.	Practical	CLO-1	CLO-2	CLO-3
1.	Basic Program Construction.	√		
2.	Getting Started: Compile & Execute a C++ Program	√		
3.	Learning data-types, Variables, Constants	√		
4.	Arithmetic Operators, Expression	√		
5.	Unary Operators (Post and Pre-Increment Decrement)	√		
6.	Escape sequences, Reserved Words and Comments.	√		
7.	Relational & Logical Operators	√		
8.	Selection logic/Conditional Logic, Conditional Jump		√	
9.	Nested IF, Nested If-Else, Switch case		√	
10.	Iterative / Repetitive Logic, Loops		√	
11.	Nested For, While, Do-while		√	
12.	Arrays, 1-Dimenion Arrays		√	
13.	2- Dimension Array, Matrices Manipulation		√	
14.	String in C++, String Operation		√	
15.	Pointers in C++,		√	
16.	Function in C++		√	
17.	, Passing parameter to a Function		√	
18.	Errors in C++, Syntactical vs Logical Errors		√	
19.	Divide by Zero, Infinite loops		√	

20.	Scope of Variables		√	
21.	Call by Reference and Call by value		√	
22.	File Handling in C++		√	
23	Understand Logical and syntactical errors			√
24	Debugging			√

Assessment Rubrics 1:

	4	**3**	**2**	**1**	**0**
Program-Planning tool: Designed Algorithm and flowcharts to create programs	The program logic is designed using well designed, Logically organized flowchart and algorithms.	The program logic is designed by neatly Designed and logically organized flow charts.	The program logic is designed using Flow chart, but Flow charts lacks a logical and visual flow of ideas, is disorganized, and is difficult to interpret.	There is no evidence of a planned design or organization of ideas in the flow chart. Work is not presentable.	No Flow Charts
Syntax: Ability to understand and follow the rules of the programming language.	Program compiles and contains no evidence of misunderstandin g or misinterpreting the syntax of the language.	Program compiles and is free from major syntactic misunderstandings, but may contain non-standard usage or superfluous elements.	Program compiles, but contains errors that signal misunderstanding of syntax – such no return 0;	Program does not compile or Contains typographical errors.	No Attempt
Logic: Ability to specify conditions, control flow, and data structures that are appropriate for the problem domain.	Program logic is correct, with no known boundary errors, and no redundant or contradictory Conditions.	Program logic is mostly correct, but may contain an occasional boundary error or redundant or Contradictory condition.	Program logic is on the right track with no infinite loops, but shows no recognition of boundary conditions (such as < vs. <=)	Program contains some conditions that specify the opposite of what is required. confuse Boolean AND/OR operators, or lead to infinite loops.	No Attempt
Correctness: Ability to code formula and algorithms that reliably produce correct answers or appropriate results.	Program produces correct answers or appropriate results for all inputs tested.	Program produces correct answers or appropriate results for most inputs.	Program approaches correct answers or appropriate results for most inputs, but can contain miscalculations in some cases.	Program does not produce correct answers or appropriate results for most inputs.	No attempt

Assessment Rubrics 2:

	Exceptional **4-6**	**Acceptable** **2-4**	**Unsatisfactory** **0-2**
Accuracy & Completion of task	The program works and meets all of the specification	The program produce correct result but doesn't display them correctly	The Program is producing incorrect results
Complexity / Readability	The code is exceptionally well organized and easy to follow.	The code is fairly easy to read.	The code is poorly organized and very difficult to read.
Creativity	The completed code adds some new element to the task beyond what was described in the task.	The completed code adds one new element to the task.	The completed code adds no new element to the task.
Knowledge and Understanding	The Completed code Demonstrate thorough understanding of the concepts	The Completed code Demonstrate moderate understanding of the concepts	The Completed code Demonstrate poor understanding of the concepts

Procedures and Policies for using Computer Lab Facility in National Sills University, Islamabad.

Computer Lab Rules:

- **BE RESPECTFUL!** Always treat the computer lab equipment AND your teacher and classmates the way that you would want your belongings and yourself to be treated.
- No food or drinks near the computers. **NO EXCEPTIONS**.
- Enter the computer lab quietly and work quietly. There are other groups and individuals who may be using the computer lab. Please be respectful.
- Surf safely! Only visit assigned websites. Some web links can contain viruses or malware. Others may contain inappropriate content. If you are not certain that a website is **SAFE**, please ask a teacher or other adult.
- Clean up your work area before you leave. All cords should be placed on the tables (not hanging off the sides). Headphones should be placed on the CPU/tower or monitor. Chair should be pushed under the tables. All trash, papers, and pencils should be picked up.
- Do not change computer settings or backgrounds.
- Ask permission before you print.
- SAVE all unfinished work to a cloud drive or jump drive. Any work that is saved to the computer will be deleted when the computer is powered off or updated at the end of the day.
- If you are the last class of the day, please POWER OFF all computers and monitors.

Practical No. 1: Basic Program Construction

Lab Objective: Introduction to Computer Programming:

Procedure:

1. Writing and editing the program.
2. Linking the program with the required library modules.
3. Compiling the program.
4. Executing the program.

Steps involved in program development:

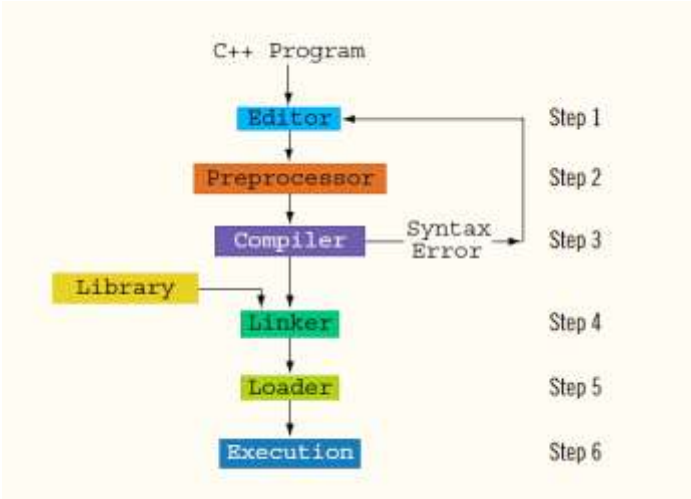
To develop the program in High level language and translate it into machine level language following steps are practiced.

1. Writing and editing the program.

2. Linking the program with the required library modules.

3. Compiling the program.

4. Executing the program.



Edit: The first phase consists of editing a file. The programmer types a C++ program with an Editor. The program source file is then stored on secondary storage device such as disk. C++ program file names often end with .cpp.

Preprocessing: The preprocessor is invoked by the compiler before the program is converted to machine language. In C++ system, a preprocessor program executes automatically before the compiler's translation phase begins. Preprocessor obeys commands called preprocessor directives.

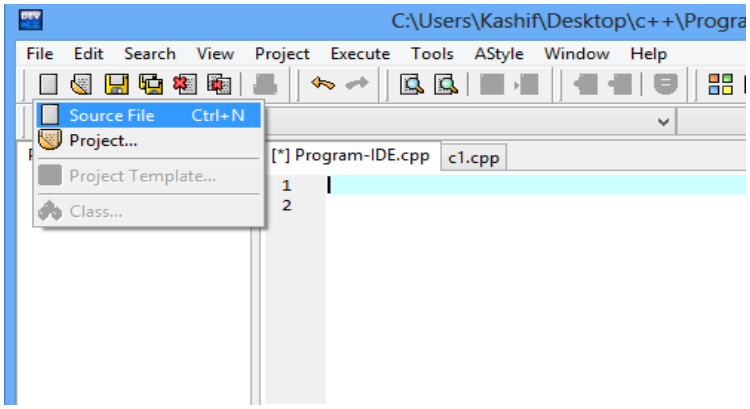
Compile: Next, the programmer gives the command to compile the program. Compiler is used to: Check that the program obeys the language rules Translate into machine language (object program).

Linking & Loading: The next phase is called linking. A linker links the object code with the code for the missing functions to produce an executable image. If the program compiles and links correctly, an executable image is produced. Loader puts program in memory.

Execute: Finally, the computer executes the program one instruction at a time. Some Integrated Development Environments (IDEs) do all this with a Build or Rebuild command.

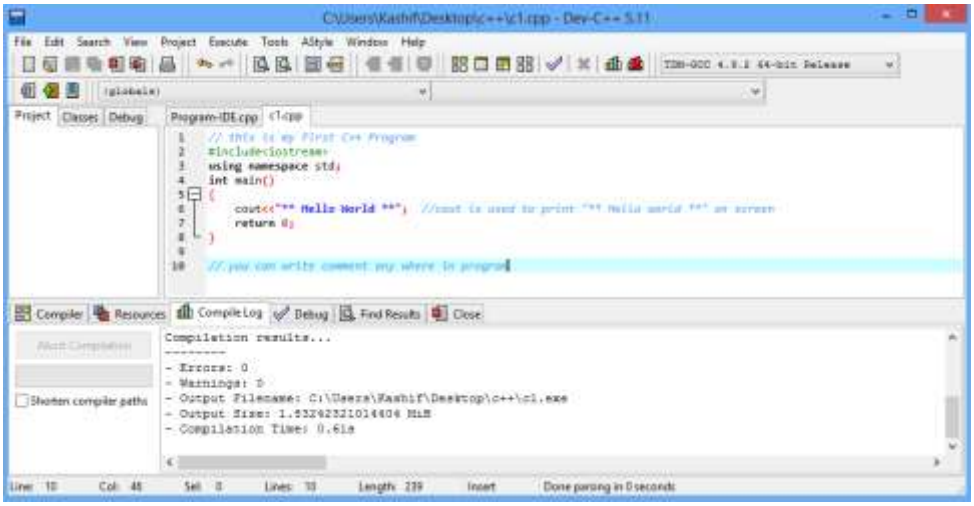
Creating a new project:

Open DevC++ from your Desktop or Programs menu. Select **"File"** from Menu bar and select option **"New".**



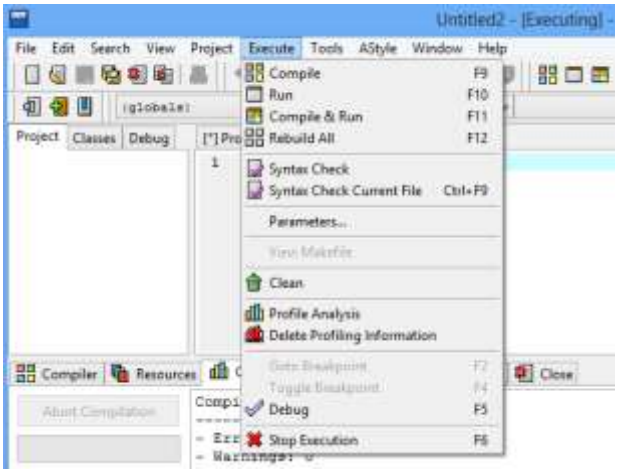
2: Write:

Write your C++ program in the Editor and Save C++ program with filename ".cpp" extension.



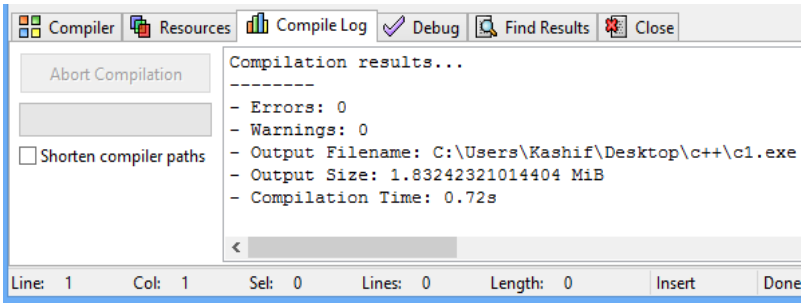
3. Compiling the code:

After writing the code, press F9, or from the Execute menu, select Compile (Building the executable file).

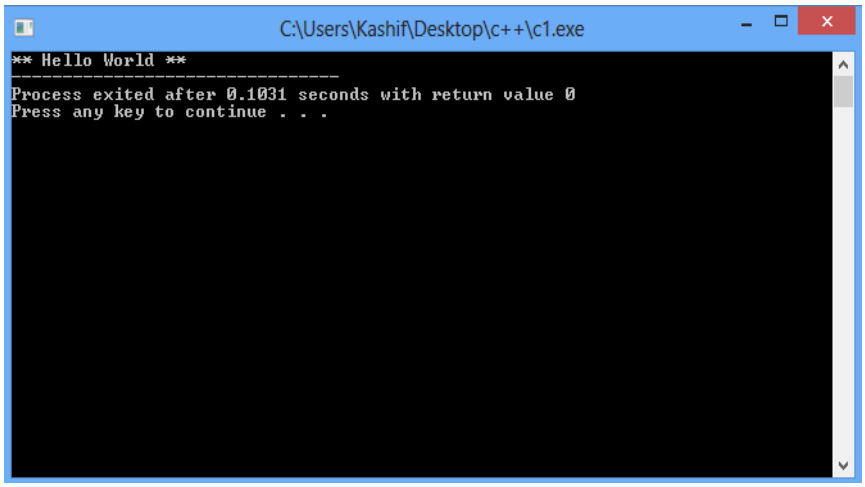


4. Build and Compile messages:

If the compilation is successful – you will see a "success" message. Else you will see the number of errors.



5. Running the exe file:



Practical No.	Checked	Marks
Lab Exercise # 1:		
Lab Exercise # 2:		
Lab Exercise # 3:		

Lab Exercise # 1:

Explore the DevC++ IDE, and resize the font size of Editor to size 14 ppt.

Lab Exercise # 2:

Confirm the translator of your Integrated Development Environment (IDE), Does it have a compiler, interpreter, or both?

Lab Exercise # 3:

Explore your C drive. Locate the program files of installed IDE. Locate include Folder and identify the header files.

Practical No.2: Getting Started: Compile & Execute a C++ Program

Lab Objective: Introduction to C++ program:

Procedure:

1. Write a simple C++ Code.
2. Learn the basic Constructs of a program
3. Compile and Execute the code.

C++ program may contain one or more sections as shown below:

```
DOCUMENTATION SECTION
LINK SECTION
DEFINITION SECTION
GLOBAL DECLARATION SECTION
Main() Function section
{
Declaration part;
Executable part;
}
```

First C++ Program:

```
//A C++ Program that print the words " Hello World. "
#include <iostream>
using namespace std;
// main() is where program execution begins.
int main() {
cout << "Hello World"; // prints Hello World
return 0;
}
```

Parts of the above program:

- The C++ language defines several headers, which contain information that is either necessary or useful to your program. For this program, the header <iostream> is needed.

- The line using namespace std; tells the compiler to use the std namespace. Namespaces are a relatively recent addition to C++.
- The next line ' // main() is where program execution begins. ' is a single-line comment available in C++. Single-line comments begin with // and stop at the end of the line.
- The line int main() is the main function where program execution begins.
- The next line cout << "Hello World"; causes the message "Hello World" to be displayed on the screen.
- The next line return 0; terminates main( )function and causes it to return the value 0 to the calling process.
- In C++, the semicolon is a statement terminator. That is, each individual statement must be ended with a semicolon. It indicates the end of one logical entity.
- Program comments are explanatory statements that you can include in the C++ code. These comments help anyone reading the source code. All programming languages allow for some form of comments. C++ supports single-line and multi-line comments. All characters available inside any comment are ignored by C++ compiler. C++ comments start with /* and end with */. A comment can also start with //, extending to the end of the line.

Alternatively the above program could be written as:

```
/* This is another way of writing a C++ Program to print Hello world on
console window */
#include<iostream>
int main()
{
        std::cout<<"Hello World ";
        return 0;
}
```

```
/* write a C++ Program to print Hello world. This is my First C++
Program on two different lines */
#include<iostream>
Using namespace std;
int main()
{
        cout<<"Hello World ";
        cout<<endl;
        cout<<"This is my First C++ Program ";
        return 0;
}
```

Practical No. 2	Checked	Marks
Lab Exercise # 1:		
Lab Exercise # 2:		

Lab Exercise # 1:

Write a C++ program to print the following lines:

When you are not practicing,

Remember,

Someone somewhere is practicing, and when you meet him he will win.

Lab Exercise # 2:

Write cout statements to print the asterisk pattern as shown below.

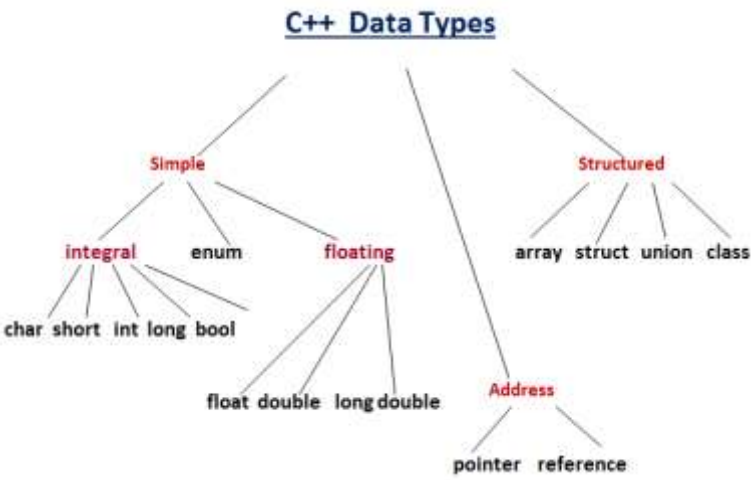
```
*
**
***
****
```

Practical N0 3: Learning data-types, variables, constants

Lab Objective: Learning data-types, Variables, Constants, Standard Input/Output.

Variables & Data Types:

A variable is a memory address where data can be stored and changed. A variable provides us with named storage that our programs can manipulate. Each variable in C++ has a specific type, which determines the size and layout of the variable's memory, this type is known as data type of that variable. The name of a variable can be composed of letters, digits, and the underscore character. It must begin with either a letter or an underscore. Upper and lowercase letters are distinct because C++ is case-sensitive.



The data type specifies the size and type of information the variable will store.

Data Type	Size	Description
int	4 bytes	Stores whole numbers, without decimals
float	4 bytes	Stores fractional numbers, containing one or more decimals. Sufficient for storing 7 decimal digits
double	8 bytes	Stores fractional numbers, containing one or more decimals. Sufficient for storing 15 decimal digits
boolean	1 byte	Stores true or false values
char	1 byte	Stores a single character/letter/number, or ASCII values

Variable Declaration in C++

A variable declaration provides assurance to the compiler that there is one variable existing with the given type and name so that compiler proceed for further compilation.

int a;

char b;

float c;

Variable Initialization in C++:

A variable can be initialized at the time of declaration or even after that. Basically initialization means storing some actual meaningful data inside the variable.

int a=2;

char b='x';

float c=2.907;

Constants:

Constants refer to fixed values that the program may not alter during its execution. These fixed values are also called **literals**. These are variables or values in C/C++ programming language which cannot be modified once they are defined.

```
//C++ Program to print an integer
#include<iostream>
using namespace std;
int main()
{
    int A;  //Variable declaration
    A=10;   //Variable Initialization
    cout<<"The Value of A is = ";
    cout<<A;
    cout<<endl;
    return 0;
}
```

Output: The Value of A is = 10

```
/* C++ Program to print Character, Floating point number, Boolean and
String */
#include<iostream>
using namespace std;
int main()
{
```

```
    char A;  //Variable declaration
    A='*';   //Variable Initialization
    float B;  //Variable of type float
    B=2.5;
    bool C;    //Boolean Variable
    C=true;
    string D;     //String Variable
    D='Pakistan'
    cout<<"The Value of A is = "<<A;
    cout<<endl;
    cout<<"The Value of B is = "<<B<<endl;
    cout<<"The Value of C is = "<<C<<endl;
    cout<<"The Value of D is = "<<D<<endl;
    return 0;
}
```

Output:

The Value of A is = *

The Value of B is = 2.5

The Value of B is = 1

The Value of B is = Pakistan

Standard Input/Output:

The standard library provides many additional ways to interact with the user via its input/output features. C++ uses a convenient abstraction called streams to perform input and output operations C++ comes with libraries that provide us with many ways for performing input and output. cin, cout, cerr, and clog are streams that handle standard inputs and standard outputs. These are stream objects defined in iostream header file.

```
cout << "Pakistan Zindabad";        // Print Pakistan Zindabad on screen

cout << 120;                        // Print Number 120 (a constant ) on screen

cout << x;                          // Print the value of x on screen
```

```
/* Program to accept values at Run Time ( Using cin>> ) and assigning it
to a variable */
#include <iostream>
using namespace std;
int main ()
{
  int i;
  cout << "Please enter an integer value :  ";
  cin >> i;
  cout << "The value you entered is =  " << i;
  return 0;
  }
```

Output: Please enter an integer value : _

The value you entered is = _

Note: Extractions on cin can also be chained to request more than one datum in a single statement.

cin >> a >> b;

Practical No. 3	Checked	Marks
Lab Exercise # 1:		
Lab Exercise # 2:		
Lab Exercise # 3:		
Lab Exercise # 4:		
Lab Exercise # 5:		
Lab Exercise # 6:		
Lab Exercise # 7:		

Lab Exercise # 1:

Execute the following C++ program and report the output.

```
//Confirms Size of each data type.
#include <iostream>
using namespace std;
int main() {
   cout << "Size of char : " << sizeof(char) << endl;
   cout << "Size of int : " << sizeof(int) << endl;
   cout << "Size of short int : " << sizeof(short int) << endl;
   cout << "Size of long int : " << sizeof(long int) << endl;
   cout << "Size of float : " << sizeof(float) << endl;
   cout << "Size of double : " << sizeof(double) << endl;
   return 0;
  }
```

Lab Exercise # 2:

Write a program that takes a character, floating point number, Boolean input at run time and display each on console window on a different line.

Lab Exercise # 3:

Write a program and confirm the following boolean variables output.

bool var1 = !true

bool var2 =false

bool var3 = true

bool = !false

Lab Exercise # 4:

Write a program and confirm the following character variables values.

Char ch1 = ‘%’

Char ch2 = 69

Lab Exercise # 5:

Write a program and confirm the following variables values.

int var1 = 3.99

float var2 = 69

float var3= 1234.4567200012345

long double var4= 1234.4567200012345

Lab Exercise # 6:

Assume a program has the following variable definitions:

int a, b = 2; float c = 4.2;

and the following statement: a = b * c;

What value will be stored in a?

Lab Exercise # 7:

Write a C++ Program to Swap Two Numbers, and print the values of Variable before and after swapping.

Practical N0.4: Arithmetic Operators, Expression

Lab Objectives: Arithmetic operators and Expression.

Operators: Operators are symbols that perform operations on variables and values. For Example , + operator to add together two values:

int x = 100 + 50;

Although the + operator is often used to add together two values, like in the example above, it can also be used to add together a variable and a value, or a variable and another variable.

int sum1 = 100 + 50; // Add to Constant 150 (100 + 50)

int sum2 = sum1 + 250; // Add a Const and Variable 400 (150 + 250)

int sum3 = sum2 + sum2; // Add two Variable 800 (400 + 400)

Arithmetic Operators: are the operators used to perform arithmetic/mathematical operations (Addition, Subtraction, Multiplication and Division) on operands. +, -, *, and / can be used with integral and floating-point data types.

Unary Operators: Operators that operate or work with a single operand are unary operators. For example: (++ , --).

X++

Binary Operators: Operators that operate or work with two operands are binary operators. For example: (+ , - , * , /).

X= A+B

```
//C++ Program to Add Two Numbers
#include<iostream>
using namespace std;
int main()
{
    int num1, num2, add;    //declared 3 variables of type integer
    num1 = 5 ;              //value of variable-1 is 5
    num2 = 3 ;               //value of variable-2 is 3
    add = num1+num2; //Add two variables and save the result in variable "add"
    cout<<endl;
```

```
    cout<<"Result = " cout<<add;
    return 0;
}
```

Operator	Meaning	Example	
+	Addition	A+B	
-	Subtraction	A-B	
*	Multiplication	A*B	
/	Division	A/B	
%	Modulus	A%B	
+=	Add and assign	A+=B	A=A+B
-=	Sub and assign	A-=B	A=A-B
*=	Multiply and Assign	A*=B	A=A*B
/=	Divide and Assign	A/=B	A=A/B
%=	Modulus and Assign	A%=	A=A%B

Expressions in C++:

An expression in C++ is an order collection of operators and operands which specifies a computation. An expression can contain zero or more operators and one or more operands, operands can be constants or variables.

```
//C++ program with a constant expression 3+2*10
#include <iostream>
using namespace std;
int main()
{
int result;                    // declaration of variable
result = 3 + 2 * 10;           //constant expression
cout<<"The result of constant expression = "
cout<<result;                  // display result value
return 0;
}
```

Output: The result of constant expression = 23

Note: Multiplication, mode and division have higher precedence than addition and subtraction.

Associativity: left to right.

Integral Expression:

```
int a=2, b=3, result;          // Declaration of Variables

result = a*a + b*b;            //Integral expression
```

Float Expression:

```
float a=2.5, b=3.6, result;    // Declaration of Variables

result = (a+b)/10;             //Float Expression
```

Practical No. 4	Checked	Marks
Lab Exercise # 1:		
Lab Exercise # 2:		
Lab Exercise # 3:		
Lab Exercise # 4:		
Lab Exercise # 5:		
Lab Exercise # 6:		
Lab Exercise # 7:		

Lab Exercise # 1:

Write a Program to Add, Subtract, Multiply and Divide two Numbers and display it on screen.

Lab Exercise # 2:

Write a Program that takes two Numbers with fractional parts at run time, Add, Subtract, Multiply and Divide them and display it on screen.

Lab Exercise # 3:

Write a program that add a constant to a character and display the result on screen. (Hint: Char ch1='B'; cout<<B+3;

Lab Exercise # 4:

Write a program that takes students numbers in six subjects at runtime and find the Percentage of Obtained Marks.

Lab Exercise # 5:

Write a program to convert temperature in degree Celsius into Fahrenheit and vice versa.

Lab Exercise # 6:

Write a program to find the circumference, area, diameter and radius of a circle.

Lab Exercise # 7:

Complete the following table by writing the value of each expression in the Value column according C++ language rules.

Expression	Values
28 / 4 – 2	
6 +12 * 2 – 8	
4+ 8 * 2	
6 + 17 % 3 -2	
2 + 22* (9 – 7)	

Practical No.5: Unary Operators (Post and Pre Increment/Decrement)

Unary operators

Unary operators are the operator that operates on single operand to give the specific result. To perform the operations on these operators we need to use only single operand. C++ provides various unary operators like unary plus operator, unary minus operator, increment operator, and decrement operator.

C++ supports two useful operators namely Increment ++ operators, Decrement – – operators. The increment operator ++ adds 1 to its operand, and the decrement operator -- subtracts 1 from its operand.

x = x+1; is the same as x++;

x = x-1; is the same as x--;

Both the increment and decrement operators can either precede (prefix) or follow (postfix) the operand.

Pre-increment: ++variable

Post-increment: variable++

Pre-decrement: --variable

Post-decrement: variable--

```
// C++ program to practice Postfix Increment Operators
#include<iostream>
using namespace std;
int main()
{
int x,i;
i=10;
x=++i;                  // Postfix increment operator
cout<<"x: "<<x;
cout<<"i: "<<i;
return 0;
}
```

Apparently seems similar, both Postfix and Prefix Operator works differently.

```
//Program to illustrate the difference b/W a++ and ++a
#include <iostream>
using namespace std;

int main() {
```

```
   int a = 21;
   int c ;

   // Value of a will not be increased before assignment.
   c = a++;
   cout << "Line 1 - Value of a++ is :" << c << endl ;

   // After expression value of a is increased
   cout << "Line 2 - Value of a is :" << a << endl ;

   // Value of a will be increased before assignment.
   c = ++a;
   cout << "Line 3 - Value of ++a is  :" << c << endl ;
   return 0;
}
```

Output:

Line 1 - Value of a++ is : 21

Line 2 - Value of a is : 22

Line 3 - Value of ++a is : 23

Practical No. 05	Checked	Marks
Lab Exercise # 1:		
Lab Exercise # 2:		

Lab Exercise # 1:

Write two separate complete programs to confirm the output of the following instruction.

```
int x = 10, y;
y = x++;
cout << y;
output: _______________
```

```
int x = 10, y;
y = ++x;
cout << y;
output: _______________
```

Lab Exercise # 2:

What will be the output of cout<<y ?

```
int x = 10,y;

y = ++x + x++ + x;
```

Practical No.6: Escape Sequences, Reserved words and Comments in C++

Escape Sequence:

Character combinations consisting of a backslash (\) followed by a letter or by a combination of digits are called "escape sequences." Escape sequences are the special characters used in control string to modify the format of the output. These characters are not displayed in output. These characters are used with combination of backslash \. This backslash \ is called escape character. The escape sequence can be inserted in any position of the string such as At the beginning of the string, In the middle of the string , At the end of the string.

For example, the escape sequence '\n' is used to insert a new line. The cursor moves from the current position on the output device to the beginning of the next line. If escape sequence '\n' is inserted at the beginning of the string then the string will be printed after printing a blank line e.g.

cout<<"\nWelcome";

cout<<"Programming\nFundamentals";

S.NO.	Escape Sequences	Purpose
1.	**\a**	Alarm (Beep, bell)
2.	**\t**	Tab
3.	**\n**	New Line
4.	**\f**	Line feed
5.	**\r**	Carriage return
6.	**\'**	Single quote mark
7.	\\\	Forward slash
8.	**\"**	Double quote mark
9.	**\b**	Backspace

```
//Program to use escape sequence character \t (tab) and \n (newline)
#include <iostream>
using namespace std;
int main() {
   // Escapr charchter \n and \t.
   cout << "Pakistan Zindabad " << "\n" ;
   cout << "Pakistan\tZindabad " << "\n" ;

   return 0;
}
```

Reserved words:

A reserved word is a word that cannot be used as an identifier, such as the name of a variable, function, or label – it is "reserved from use". Reserved words in C++ library are used to perform an internal operation.

They are used by the language, these keywords are not available for re-definition .There are a total of 95 reserved words in C++.

Exclusive reserved list:

Following list of reserved/ keyword cannot be used as variable or function names.

double float int short unsigned break continue else for long signed switch
case default goto sizeof char do if return while namespace bool false using true
not cin endl main std cout include iostream string cout

int cout // variable name cout isn't allowed

Comments:

They help the person writing a program, and anyone else who must read the source file, understand what's going on. The compiler ignores comments. Two styles of comments provided in c++, one line and multiline comments.

One line Comments: Starts with // and proceeds to end of line

```
// This is an example of a comment.
```

Multiline comment: Starts with /* and proceeds to first */

```
/* This is another example of a comment. */

/* This is an example of a comment that

spans

multiple lines. */
```

Practical No. 06	Checked	Marks
Lab Exercise # 1:		
Lab Exercise # 2:		

Lab Exercise # 1:

Write a program to having following given commands. Report the output.

```
cout<<" \" COMPUTER SCIENCE \" ";

cout<<" COMPUTER\nSCIENCE ";

cout<<" COMPUTER\tSCIENCE ";

cout<<" COMPUTER\vSCIENCE ";

cout<<" COMPUTER\bSCIENCE ";

cout<<" COMPUTER\aSCIENCE ";

cout<<" COMPUTER\rSCIENCE ";

cout<<" \\\ COMPUTER SCIENCE \\\ ";

cout<<" COMPUTER SCIENCE \? ";
```

Lab Exercise # 2:

Write a program to display the following output using a single cout statement.

```
Subject        Marks

Mathematics     90

Computer        77

Chemistry       69
```

Practical No.7 Relational & Logical Operators

Relational Operators:

These are used for the comparison of the values of two operands. For example, checking if one operand is equal to the other operand or not, an operand is greater than the other operand or not, etc. Some of the relational operators are (>, < , ==, >= , <= , !=).

a > b // checks if a is greater than b

Operator	Description	Example
<	Less then	A<B
>	Greater then	A>B
==	Equals to	A==B
<=	Less then	A<=B
>=	Greater then	A>=B
!=	Equals to	A!=B

Logical Operators:

Logical Operators are used to combine two or more conditions/constraints or to complement the evaluation of the original condition in consideration.

Logical AND && (Expression 1 and Expression 2)

Logical OR || (Expression 1 OR Expression 2)

Logical Not ! (Not Expression)

Logical Operators are used to combine two or more conditions/constraints or to complement the evaluation of the original condition in consideration.

The result of the operation of a logical operator is a Boolean value either true or false. For example, the logical AND represented as '&&' operator in C++ returns true when both the conditions under consideration are satisfied. Otherwise, it returns false. Therefore, A && B returns true when both A and B are true (i.e. non-zero).

Input	*Output*
A	**not A**
false	true
true	false

For Logical Operator AND the conditions are evaluated using below given truth table.

Truth table		
X	Y	X AND Y
False	False	False
False	True	False
True	False	False
True	True	True

Truth table		
X	Y	X AND Y
0	0	0
0	1	0
1	0	0
1	1	1

For Logical Operator OR the conditions are evaluated using below given truth table.

Input		*Output*
A	B	**A or B**
false	false	false
false	true	true
true	false	true
true	true	true

```
//Logical Operators (&&)
#include <iostream>
using namespace std;
int main()
{
 int x ;         //Declaration
 cout<<"Enter a value for variable x = ";
 cin>>x;        // x will takes on a value at run time
 cout<<endl;
  // here we have used two relational and one logical Operators
 if((x != 5) && (x > 0))
 {
    cout <<"Successfully compared";
 }
}
```

Practical No. 07	Checked	Marks
Lab Exercise # 1:		
Lab Exercise # 2:		

Lab Exercise # 1:

Execute following program and interpret the output.

```
#include<iostream>
Using namespace std;
int main()
{
  int X1=-1, X2=1, Y1, Y2;
  Y1 = !X1 && X2;
  Y2 = !X1 || X2;
  cout<<" Y1 = "<<Y1<<endl;
  cout<<" Y2 = "<<Y2<<endl;
  return 0;
}
```

Lab Exercise # 2:

Write a C++ Program that uses the logical AND operator to avoid division by zero: (Hint: (y != 0) && (x / y)

Practical No.8. Selection logic, or Conditional Logic, Conditional Jump

Control structure:

Control Structures are just a way to specify flow of control in programs. It basically analyzes and chooses in which direction a program flows based on certain parameters or conditions. There are three basic types of logic, or flow of control, known as:

1. Sequence logic, or sequential flow
2. Selection logic, or conditional flow
3. Iteration logic, or repetitive flow

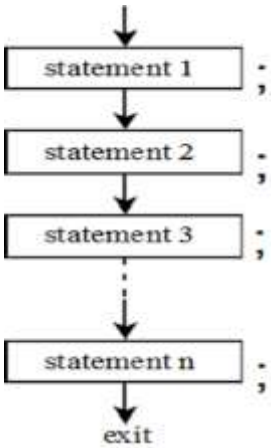
Sequence logic, or sequential flow:

As name refers instructions are executed in the sequence in which they are written in the program

Sequence is the default control structure; instructions are executed one after another.

Example: Compound Statement:

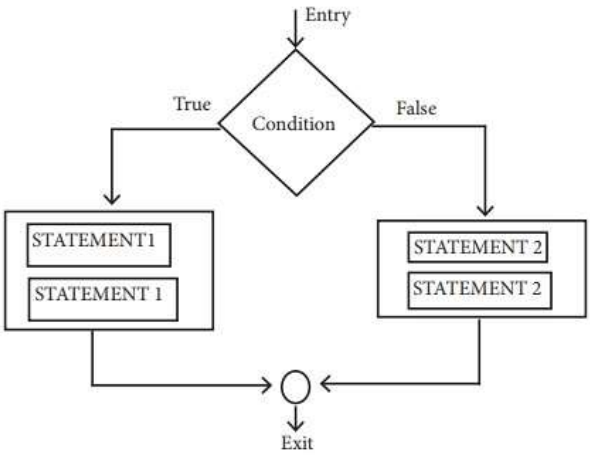
```
{

    Statements

    Statements - 1

    Statements - 2

    - - - - - - - - -

 }
```



Selection logic or conditional flow:

A structure which select which statement or block of statements will execute on the basis of our programming logic.

The conditional IF-THEN or IF-THEN-ELSE control structure allows a program to follow alternative paths of execution.



C++ has the following conditional statements:

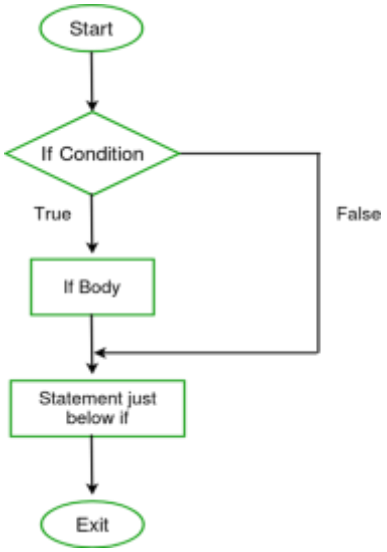
1. Use if to specify a block of code to be executed, if a specified condition is true
2. Use else to specify a block of code to be executed, if the same condition is false
3. Use else if to specify a new condition to test, if the first condition is false
4. Use switch to specify many alternative blocks of code to be executed

The if Statement:

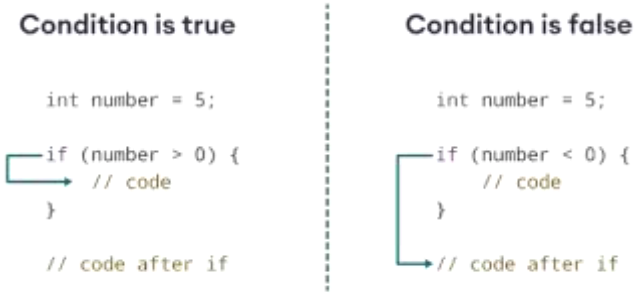
if statement is the most simple decision-making statement. It is used to decide whether a certain statement or block of statements will be executed or not i.e if a certain condition is true then a block of statement is executed otherwise not. A block of C++ code is executed only if a condition is true.

Syntax

```
if (condition)
{
  // block of code to be executed if the condition is true
}
```



Working of if Statement:



1. The if statement evaluates the condition inside the parentheses ( ).
2. If the condition evaluates to true, the code inside the body of if is executed.
3. If the condition evaluates to false, the code inside the body of if is skipped.
4. Note that if is in lowercase letters. Uppercase letters (If or IF) will generate an error.

```
// C++ program to illustrate If statement
#include <iostream>
using namespace std;
int main()
{
    int i = 10;
    if (i < 15)
    {
        cout << "10 is less than 15 ";
    }
    cout << endl<<"I am Not in if";
    return 0;
}
```

Output: 10 is less than 15

I am Not in if

Multiple If:

In case of two different conditions we can write multiple if. The condition is checked before executing subsequent statement.

```
// C++ program to illustrate multiple If statement
#include <iostream>
using namespace std;
int main()
{
    int i;
    cout<<"Enter any positive Number = ";
```

```
    cin>>i;
    if (i < 15)
    {
        cout << "Number you entered  is less than 15 ";
    }
    if (i > 15)
    {
        cout << " Number you entered is greater than 15 ";
    }

    cout << endl<<"I am Not in any of the if";
    return 0;
}
```

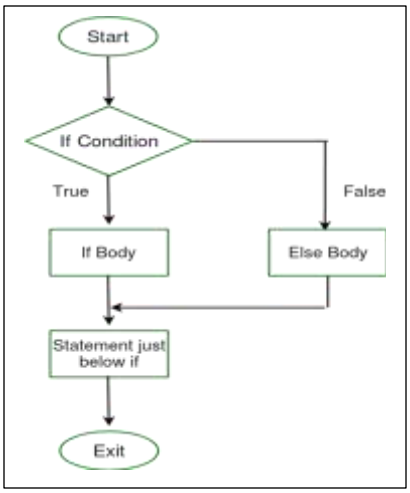
To ignore the compound statements should be enclosed inside the body of the if.

```
// C++ program to illustrate multiple If statement
#include <iostream>
using namespace std;
int main()
{
    int i;
    cout<<"Enter any positive Number = ";
    cin>>i;
    if (i < 15)
    {
        cout << "Number you entered  is less than 15 ";
        cout<<endl;
        cout << "This line is ignored if the condition is false
                 along-side the line above. ";
    }

    cout << endl<<"I am Not in any of the if";
    return 0;
}
```

If-else:

The if statement alone tells us that if a condition is true it will execute a block of statements and if the condition is false it won't. We can use the else statement with if statement to execute a block of code when the condition is false. The if statement can have an optional else clause. Its syntax is:

```
if (condition) {

   // block of code if condition is true

}

else {

   // block of code if condition is false

}
```



Working of if-Else Statement:

If the condition evaluates true,

i. The code inside the body of if is executed
ii. The code inside the body of else is skipped from execution

If the condition evaluates false,

i. The code inside the body of else is executed
ii. The code inside the body of if is skipped from execution

Program:

```
// Program to understand working of if-else Statement
#include<iostream>
Using namespace std;
int main()
{
  int number;
  cout<<"enter any Number";
  cin>>number;
    if (number > 0)
        cout << "The number is positive." << endl;
    else
        cout << "The number is negative." << endl;
   return 0;
}
```

In case if multiple statements (Compound statements) has to execute or skipped it must be enclosed inside the body of if.

Note: Body i.e {} are optional in if , If-Else

```
// Program to understand working of if-else Statement
#include<iostream>
Using namespace std;
int main()
{
  int number;
  cout<<"enter any Number";
  cin>>number;
    if (number > 0)
        {
           cout << "The number is positive." << endl;
           cout<< "The condition is evaluated as true";
         }
    else
        {
         cout << "The number is negative." << endl;
         cout<< "the condition is evaluated as false";
        }
   return 0;
}
```

Practical No. 08	Checked	Marks
Lab Exercise # 1:		
Lab Exercise # 2:		
Lab Exercise # 3:		

Lab Exercise # 1:

Write a program to print positive number entered by the user. If the user enters negative number print number entered is positive otherwise print number is negative.

Lab Exercise # 2:

Write a Program that takes input at run time and check whether an integer is odd or even.

Lab Exercise # 3:

Input at run time: Marks

Process : If mark greater than and equal to 75, score will be A If mark less than 75 and greater than and equal to 60, score will be B If mark less than 60 and greater than and equal to 45, score will be C If mark less than 30, score will be D

Output: Print the grade of the score.

Practical No. 9 Nested IF, Nested If-Else, Switch statements

Nested If:

Nested if statements mean an if statement inside another if statement. When there is more than one condition and they are dependent on one another, then if statement can be nested. The nested if statement allows checking multiple conditions one after another.

```
//Nested If
#include <iostream>
using namespace std;
int main(){
   int num=90;
   /* Nested if statement. An if statement
    * inside another if body
    */
   if( num < 100 ){
      cout<<"number is less than 100"<<endl;
      if(num > 50){
         cout<<"number is greater than 50";
      }
   }
   return 0;
}
```

Nested If-Else:

Nested if statements mean an if statement inside another if statement. Sometimes, we need to use an if statement inside another if statement. This is known as nested if statement.

Think of it as multiple layers of if statements. There is a first, outer if statement, and inside it is another, inner if statement.

```
//Nested If-Else-If
#include <iostream>
using namespace std;
int main(){
   int num;
   cout<<"Enter an integer number between 1 & 99999: ";
   cin>>num;
   if(num <100 && num>=1) {
      cout<<"Its a two digit number";
   }
   else if(num <1000 && num>=100) {
      cout<<"Its a three digit number";
```

```
   }
   else if(num <10000 && num>=1000) {
      cout<<"Its a four digit number";
   }
   else if(num <100000 && num>=10000) {
      cout<<"Its a five digit number";
   }
   else {
      cout<<"number is not between 1 & 99999";
   }
   return 0;
}
```

if...else...else if statement:

The if...else statement is used to execute a block of code among two alternatives. However, if we need to make a choice between more than two alternatives, we use the if...else if...else statement.

Here, a user can decide among multiple options. The C++ if statements are executed from the top down. As soon as one of the conditions controlling the if is true, the statement associated with that if is executed, and the rest of the C++ else-if ladder is bypassed. If none of the conditions are true, then the final else statement will be executed.

Switch:

Switch case statement is used when we have multiple conditions and we need to perform different action based on the condition. When we have multiple conditions and we need to execute a block of statements when a particular condition is satisfied. In such case either we can use lengthy if..else-if statement or switch case. The problem with lengthy if..else-if is that it becomes complex when we have several conditions. The switch case is a clean and efficient method of handling such scenarios.

```
//Program for switch
#include <iostream>
using namespace std;
int main(){
int day = 2;
switch (day ) {
case  1:
    cout << "Saturday";
case  2:
    cout << "Sunday"; }
return 0;
}
```

Break:

Break statements are used when you want your program-flow to come out of the switch body. Whenever a break statement is encountered in the switch body, the execution flow would directly come out of the switch, ignoring rest of the cases. This is why we must end each case block with the break statement.

```
//Program for switch with break
#include <iostream>
using namespace std;
int main(){
   char ch='b';
   switch(ch) {
      case 'd': cout<<"Case1 ";
      break;
      case 'b': cout<<"Case2 ";
      break;
      case 'x': cout<<"Case3 ";
      break;
      case 'y': cout<<"Case4 ";
      break;
   }
   return 0;
}
```

Default:

A switch statement can have an optional default case, which must appear at the end of the switch. The default case can be used for performing a task when none of the cases is true. No break is needed in the default case.

```
//Program for switch with break and default
#include <iostream>
using namespace std;
int main(){
int day = 4;
switch (day) {
  case 6:
    cout << "Today is Saturday";
    break;
  case 7:
    cout << "Today is Sunday";
    break;
  default:
    cout << "Looking forward to the Weekend";
}
```

Practical No. 09	Checked	Marks
Lab Exercise # 1:		
Lab Exercise # 2:		
Lab Exercise # 3:		

Lab Exercise # 1:

Write a program which input three numbers and display the largest number. Find Largest Number Using Nested if...else statement.

Lab Exercise # 2:

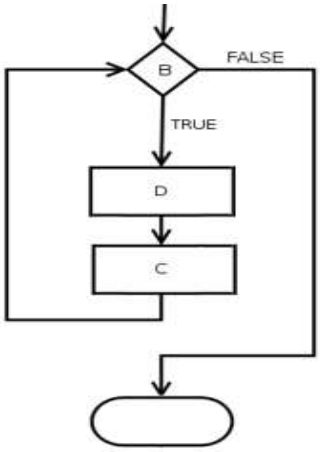
Write a program that print total number of days in a month using switch case.

Lab Exercise # 3:

Write a C++ Program to Make a Simple Calculator to Add, Subtract, Multiply or Divide Using Switch...case.

Practical No. 10, 11: Repetitive/ Iterative Logic, Loops

In general, statements are executed sequentially: The first statement in a function is executed first, followed by the second, and so on. There may be a situation, when you need to execute a block of code several numbers of times. A loop statement allows us to execute a statement or group of statements multiple times.



Loop:

C++ programming language provides the following type of loops to handle looping requirements.

1. **For Loop:** Execute a sequence of statements multiple times and abbreviates the code that manages the loop variable.
2. **While Loop:** Repeats a statement or group of statements while a given condition is true. It tests the condition before executing the loop body.
3. **do-While:** Like a 'while' statement, except that it tests the condition at the end of the loop body.
4. **Nested Loop:** Using one or more loop inside any another 'while', 'for' or 'do.. while' loop.

For Loop:

When you know exactly how many times you want to loop through a block of code, use the for loop.

```
for (initialization; condition; update) {

   // body of-loop

}
```

1. **Initialization:** is executed (one time) before the execution of the code block.
2. **Condition:** defines the condition for executing the code block. if true, the body of for loop is executed if false, the for loop is terminated

3. **Update:** is executed (every time) after the code block has been executed. updates the value of initialized variables and again checks the condition

```
Example
for (int i = 0; i < 5; i++)
 {
  cout << i << "\n";
 }
```

Explanation

1. Statement 1: sets a variable before the loop starts (int i = 0).
2. Statement 2: defines the condition for the loop to run (i must be less than 5). If the condition is true, the loop will start over again, if it is false, the loop will end.
3. Statement 3: Increases a value (i++) each time the code block in the loop has been executed.

Example:

```
// C++ Program to display a text 5 times
#include <iostream>
using namespace std;

int main() {
    for (int i = 1; i <= 5; ++i) {
        cout <<  "Hello World! " << endl;
    }
    return 0;
}
```

```
// C++ Program for Printing Numbers From 1 to 5
#include <iostream>
using namespace std;

int main() {
        for (int i = 1; i <= 5; ++i) {
        cout << i << " ";
    }
    return 0;
}
```

Loop with a different incrimination i.e. instead of fix one incrimination the loop counter is increased as i = i+2;

```
// The loop counter is increased as i = i+2 rather than fix i++
```

```
//Using for loops
#include <iostream>
using namespace std;
int main() {
    for (int i = 0; i <= 10; i+=2)
    cout << "Value of i = " << i << endl;
    return 0;
}
```

Infinite loop:

```
//C++ program to demonstrate infinite loops
// using for
#include <iostream>
using namespace std;
int main ()
{
int i;
for ( ; ; ) // This is an infinite for loop as the condition….expression
is blank
    {
        cout << "This loop will run forever.\n";
    }
    // This is an infinite for loop as the condition
return 0;
}
```

While Loop:

A loop is mainly consisted of three statements – initialization expression, test expression, update expression. The syntax of the three loops – For, while and do while mainly differs on the placement of these three statements.

```
initialization expression;
while (test_expression)
{
   // statements-1
  // statements-2
   update_expression;
}
```

```
// C++ program to illustrate while loop
#include <iostream>
using namespace std;
int main(){
```

```
    int i = 1;            // initialization expression
    while (i < 6)      // test expression
    {
        cout << "Hello World\n";
         i++;             // update expression
}
    return 0;
}
```

Do-While Loop:

In some situations it is necessary to execute body of the loop before testing the condition. Such situations can be handled with the help of do-while loop. do statement evaluates the body of the loop first and at the end, the condition is checked using while statement. General format of do-while loop is,

```
initialization expression;
do
{
   // statements
   update_expression;
} while (test_expression);
```

Example Program:

```
#include <iostream>
using namespace std;
int main()
{
    int i = 2;                   // Initialization expression
    do
    {
        // loop body
        cout << "Hello World\n";
        i++;                        // update expression
    }
    while (i < 1);                // test expression
    return 0;
}
```

Nested For Loop:

We can also have nested for loop, i.e. one for loop inside another for loop. Basic syntax is,

```
for(initialization; condition; increment/decrement)
{
    for(initialization; condition; increment/decrement)
    {
        statement;
    }
}
```

Jumping out of a loop:

Sometimes, while executing a loop, it becomes necessary to skip a part of the loop or to leave the loop as soon as certain condition becomes true, that is jump out of loop.

break statement:

When break statement is encountered inside a loop, the loop is immediately exited and the program continues with the statement immediately following the loop.

Continue statement:

It causes the control to go directly to the test-condition and then continue the loop process. On encountering continue, cursor leave the current cycle of loop, and starts with the next cycle.

Practical No. 10,11	Checked	Marks
Lab Exercise # 1:		
Lab Exercise # 2:		
Lab Exercise # 3:		
Lab Exercise # 4:		
Lab Exercise # 5:		
Lab Exercise # 6:		
Lab Exercise # 7:		
Lab Exercise # 8:		

Lab Exercise # 1:

Write a C++ program to Print Table of any Number.

Lab Exercise # 2:

Write a method with a for loop that computes the sum of first n positive integers: sum = 1 + 2 + 3 + … + n.

Lab Exercise # 3:

Write a program to print natural numbers in reverse from n to 1 using while loop

Lab Exercise # 4:

Write a program to print all even numbers between i to n using while loop.

Lab Exercise # 5:

Write a C++ program that will print the following pattern using for Loop.

```
*
**
***
****
******
*******
```

Lab Exercise # 6:

Write a program to print natural numbers in reverse from n to 1 using while loop.

Lab Exercise # 7:

Write a program to print all even numbers between i to n using while loop.

Lab Exercise # 8:

Write a C++ program that will print the following pattern using for Loop.

```
1 2 3 4

1 2 3

1 2

1
```

Practical No. 12, 13. Arrays in C++, 1-D & 2-D Arrays

Arrays:

An array is a series of elements of the same type placed in contiguous memory locations that can be individually referenced by adding an index to a unique identifier. That means that, for example, Six values of type int can be declared as an array without having to declare 6 different variables (each with its own identifier). Instead, using an array, the Six int values are stored in contiguous memory locations, and all six can be accessed using the same identifier, with the proper index. For example, an array containing 6 integer values of type int called X could be represented as:

Declaring Arrays:

Like other variables in C++, an array must be defined before it can be used to store information. Like other definitions, an array definition specifies a variable type and a name. But it includes another feature i.e. size.

int score[5];

This is like declaring 5 variables of type int:

score[0], score[1], … , score[4]

Access Elements in C++ Array:

- Elements of an array can be accessed by indices.
- The array indices start with 0. Meaning x[0] is the first element stored at index 0.
- If the size of an array is n, the last element is stored at index (n-1). In this example, x[5] is the last element**.**

Initializing array in Declarations:

To initialize an array, you have to specify a list of initial values for the array elements, separate them with commas and enclose the list within braces.

int array1[5] = {23, 10, 16, 37, 12};

int array1[ ] = { 23, 10, 16, 37};

Method 2: Alternatively an array can be statically initialized as.

int arr[2];

arr[0] = 19;

arr[1] = 10;

```
// statically initialize an array and Print array elements using for loop
#include <iostream>
using namespace std;
int main() {
    int numbers[5] = {7, 5, 6, 12, 35};
   cout << "\n The numbers are: ";
   for (int i = 0; i < 5; ++i) {
        cout << numbers[i] << "  ";
    }
    return 0;
}
```

```
//  To initialize array at run-time
#include <iostream>
using namespace std;
int main() {
    int numbers[5];
    cout << "Enter 5 numbers: " << endl;
    //  store input from user to array
    for (int i = 0; i < 5; ++i) {
        cin >> numbers[i];
    }
return 0;
}
```

Types of Arrays:

In the C++ programming language, we do have mainly two types of Array: Single-Dimensional Arrays and Multi-dimensional Arrays. The **single-Dimensional** stores values hold the values in the form of list, while the **Multi-dimensional** Array stores the value in the matrix.

Specifically The types of Array provided by C++ are follows:

1. Single Dimensional Array (1-D Array)

2. Two Dimensional Array (M-D Array)

3. Three Dimensional array (M-D Array)

4. Character Array or Strings.

2-D Arrays or Matrices:

The Two Dimensional array is used for representing the elements of the array in the form of the rows and columns and they are used for representing the Matrix. A Two Dimensional Array uses the two subscripts for declaring the elements of the Array. For example

int a[3][3]

The first 3 represents the total number of Rows and the Second 3 represents the Total number of Columns.

2-D Array can be visualized as an array of arrays. The image below depicts a two-dimensional array

	Col1	Col2	Col3	Col4
Row1	Arr[0][0]	Arr[0][1]	Arr[0][2]	Arr[0][3]
Row2	Arr[1][0]	Arr[1][1]	Arr[1][2]	Arr[1][3]
Row3	Arr[2][0]	Arr[2][1]	Arr[2][2]	Arr[2][3]
Row4	Arr[3][0]	Arr[3][1]	Arr[3][2]	Arr[3][3]
⋮				

Initializing a 2D array in C++:

int arr[4][2] = {{1234, 56}, {1212, 33}, {1434, 80}, {1312, 78} } ;

int arr[4][3]={{1,2,3}, {2,3,4}, {3,4,5}, {4,5,6}};

Printing 2-D array:

```
// Printing 2-D array
int i,j;
    cout<<"Printing a 2D Array:\n";
    for(i=0;i<4;i++)
    {
        for(j=0;j<2;j++)
        {
            cout<<"\t"<<arr[i][j];
        }
        cout<<endl;
    }
}
```

Practical No. 12,13	Checked	Marks
Lab Exercise # 1:		
Lab Exercise # 2:		
Lab Exercise # 3:		
Lab Exercise # 4:		

Lab Exercise # 1:

Write a C++ program that defines a 1-D Array. Initialize it at run time and print the stored values using while loop.

Lab Exercise # 2:

Write a C++ program to calculate average and percentage marks of a student.

Lab Exercise # 3:

Write a C++ Program to Find Largest Element of an Array.

Lab Exercise # 4:

Write a C++ program that two multiply 2x2 Matrices using for Loop.

Practical No. 14. Pointers in C++

Pointers in C++:

In C++, a pointer refers to a variable that holds the address of another variable. Like regular variables, pointers have a data type. For example, a pointer of type integer can hold the address of a variable of type integer. A pointer of character type can hold the address of a variable of character type.

Addresses in C++:

To understand C++ pointers, you must understand how computers store data. When you create a variable in your C++ program, it is assigned some space the computer memory. The value of this variable is stored in the assigned location. To know the location in the computer memory where the data is stored, C++ provides the & (reference) operator. The operator returns the address that a variable occupies. For example, if x is a variable, &x returns the address of the variable.

Pointer Declaration Syntax

The declaration of pointer in C++ takes the following syntax:

datatype *variable_name;

1. The datatype is the base type of the pointer which must be a valid C++ data type.
2. The Variable_name is should be the name of the pointer variable.
3. Asterisk used above for pointer declaration is similar to asterisk used to perform multiplication operation. It is the asterisk that marks the variable as a pointer.

Reference operator (&) and Deference operator (*)

The reference operator (&) returns the variable's address. The dereference operator (*) helps us get the value that has been stored in a memory address.

For example:

If we have a variable given the name num, stored in the address 0x234 and storing the value 28.

1. The reference operator (&) will return 0x234.
2. The dereference operator (*) will return 5.

Printing Variable Addresses in C++:

```
#include <iostream>
using namespace std;
int main()
```

```
{
int var1 = 3;              // declare variables
int var2 = 24;
int var3 = 17;
cout<<"Address of var1: "<< &var1 << endl;   // print address of var1
cout<< "Address of var2: "<< &var2 <<endl;  // print address of var2
cout<< "Address of var3: "<< &var3 <<endl;   // print address of var3
return 0;
}
```

Assigning Values to pointer variable:

```
#include<iostream>
int main(){
int* P_Var, var;
var = 5;
P_Var = &var;     // assign address of var to P_Var

// access value pointed by P_Var

cout << *P_Var << endl;   // Output: 5
return 0;
}
```

Changing Value Pointed by Pointers:

```
#include<iostream>
int main(){
int var = 5;
int* P_Var;

P_Var = &var;        // assign address of var
// change value at address P_Var
*P_Var = 1;
cout << var << endl;               // Output: 1
return 0;
}
```

Practical No. 14	Checked	Marks
Lab Exercise # 1:		
Lab Exercise # 2:		
Lab Exercise # 3:		

Lab Exercise # 1:

Write a C++ program to accept the set of 3 numbers and print the numbers using pointer.

Lab Exercise # 2:

Write a C++ program to display the multiplication of two numbers using pointer.

Lab Exercise # 3:

Write a C++ program to swap two numbers using pointer.

Practical No. 15. Functions in C++

Function in C++:

A function is a block of code that performs a specific task which only runs when it is called. A function is a group of statements that together perform a task. . There are two types of function:

1: Standard Library Functions: Predefined in C++

2: User-defined Function: Created by users

Defining a Function

A C++ function definition consists of a function header and a function body.

Return Type − A function may return a value. The return_type is the data type of the value the function returns. Some functions perform the desired operations without returning a value. In this case, the return_type is the keyword void.

Function Name − this is the actual name of the function. The function name and the parameter list together constitute the function signature.

Parameters − A parameter is like a placeholder. When a function is invoked, you pass a value to the parameter. This value is referred to as actual parameter or argument. The parameter list refers to the type, order, and number of the parameters of a function. Parameters are optional; that is, a function may contain no parameters.

Function Body − The function body contains a collection of statements that define what the function does.

For example:

If we have a variable given the name num, stored in the address 0x234 and storing the value 28.

3. The reference operator (&) will return 0x234.
4. The dereference operator (*) will return 5.

Display a Text:

```
//User defined function in C++
#include <iostream>
using namespace std;
// declaring a function
void greet() {
    cout << "Hello there!";
}
```

```
int main() {
    // calling the function
    greet();
    return 0;
}
```

Function with Parameters :

```
// Function with parameters
#include <iostream>
using namespace std;
// display a number
void displayNum(int n1, float n2) {
    cout << "The int number is " << n1;
    cout << "The double number is " << n2;
}

int main() {
     int num1 = 5;
     double num2 = 5.5;
    // calling the function
    displayNum(num1, num2);

    return 0;
}
```

Function Prototype

In C++, the code of function declaration should be before the function call. However, if we want to define a function after the function call, we need to use the function prototype.

```
 // function prototype
void add(int, int);

int main() {
    // calling the function before declaration.
    add(5, 3);
    return 0;
}
// function definition
void add(int a, int b) {
    cout << (a + b);
}
```

Parameters and Arguments

Information can be passed to functions as a parameter. Parameters act as variables inside the function.

Parameters are specified after the function name, inside the parentheses. You can add as many parameters as you want, just separate them with a comma:

```
void myFunction(string fname)
{
  cout << fname << " Khan\n";
}

int main() {
  myFunction("Ali");
  myFunction("Asslam");
  myFunction("Ayesha");
  return 0;
}

// Ali Khan
// Asslam Khan
// Ayesha Khan
```

Multiple Parameters

Inside the function, you can add as many parameters as you want

```
void myFunction(string fname, int age) {
  cout << fname << " Khan. " << age << " years old. \n";
}

int main() {
  myFunction("Ali", 3);
  myFunction("Asslam", 14);
  myFunction("Ayesha", 30);
  return 0;
}

// Ali Khan 3 years old.
// Asslam Khan 14 years old.
// Ayesha Khan 30 years old.
```

Practical No. 15	Checked	Marks
Lab Exercise # 1:		
Lab Exercise # 2:		
Lab Exercise # 3:		
Lab Exercise # 4:		

Lab Exercise # 1:

Write a C++ program that adds two integers using a function “sum”.

Lab Exercise # 2:

Write a C++ program that add two integers using a function “ sum” and pass the numbers as parameter to the user defined functions.

Lab Exercise # 3:

Write a C++ program to convert temperature from Fahrenheit to Centigrade using function.

Lab Exercise #4:

Write a C++ Program to calculate Average of 5 subjects and find percentage using function.

Practical No. 16. Function Parameters

Pass By Reference

C++, we can pass parameters to a function either by pointers or by reference. Call by reference method copies the reference of an argument into the formal parameter. Inside the function, the reference is used to access the actual argument used in the call. This means that changes made to the parameter affect the argument.

Display a Text:

```
#include <iostream>
using namespace std;
void swapNums(int &x, int &y) {
  int z = x;
  x = y;
  y = z;
}

int main() {
  int firstNum = 10;
  int secondNum = 20;

  cout << "Before swap: " << "\n";
  cout << firstNum << secondNum << "\n";
  // Call the function, which will change the values of firstNum and
secondNum
  swapNums(firstNum, secondNum);
  cout << "After swap: " << "\n";
  cout << firstNum << secondNum << "\n";
  return 0
```

Pass Arrays as Function Parameters:

You can also pass arrays to a function

```
// program to print Array Elements
#include <iostream>
using namespace std;
void myFunction(int myNumbers[5]) {
  for (int i = 0; i < 5; i++) {
    cout << myNumbers[i] << "\n";
  }
}
```

```
int main() {
  int myNumbers[5] = {10, 20, 30, 40, 50};
  myFunction(myNumbers);
  return 0;
}    // calling the function
    displayNum(num1, num2);

    return 0;
}
```

Function Prototype

Example Explained

The function (myFunction) takes an array as its parameter (int myNumbers[5]), and loops through the array elements with the for loop.

When the function is called inside main(), we pass along the myNumbers array, which outputs the array elements.

Note that when you call the function, you only need to use the name of the array when passing it as an argument myFunction(myNumbers). However, the full declaration of the array is needed in the function parameter (int myNumbers[5]).

Return Values

The void keyword, used in the previous examples, indicates that the function should not return a value. If you want the function to return a value, you can use a data type (such as int, string, etc.) instead of void, and use the return keyword inside the function:

```
#include<iostream>
Using namespace std;
// function prototype

int myFunction(int x, int y) {
  return x + y;
}

int main() {
  cout << myFunction(5, 3);
  return 0;
}
```

Practical No. 16	Checked	Marks
Lab Exercise # 1:		
Lab Exercise # 2:		

Lab Exercise # 1:

Write a C++ program to swap two numbers using pass by pointers and call by reference. Compare and contrast both program.

Lab Exercise # 2:

Write a C++ program to demonstrate differences between pointer and reference

Practical No. 17. Scope of Variables in C++

A scope is a region of the program and broadly speaking there are three places, where variables can be declared

1. Inside a function or a block which is called local variables,
2. In the definition of function parameters which is called formal parameters.
3. Outside of all functions which are called global variables.

Local variables can be used only by statements that are inside that function or block of code. Local variables are not known to functions on their own.

```
#include <iostream>
using namespace std;
int main () {
   // Local variable declaration:
   int a, b;
   int c;

   // actual initialization
   a = 10;
   b = 20;
   c = a + b;

   cout << c;
   return 0;
}
```

Global Variable:

Global variables are defined outside of all the functions, usually on top of the program. The global variables will hold their value throughout the lifetime of your program. A global variable can be accessed by any function.

```
// program to print a text
#include <iostream>
using namespace std;
// Global variable declaration:
int g;
int main () {
   // Local variable declaration:
```

```
   int a, b;

   // actual initialization
   a = 10;
   b = 20;
   g = a + b;

   cout << g;
   return 0;
}
```

Access Global and local occurrence of the same variable

A program can have the same name for local and global variables but the value of a local variable inside a function will take preference. For accessing the global variable with same name, you'll have to use the scope resolution operator.

```
// Access Global and local occurrence of the same variable
#include <iostream>
using namespace std;
// Global variable declaration:
int g = 20;
int main () {
   // Local variable declaration:
   int g = 10;

   cout << g;   // Local
   cout << ::g; // Global
   return 0;
}
```

Practical No. 17	Checked	Marks
Lab Exercise # 1:		
Lab Exercise # 2:		

Lab Exercise # 1:

Write a C++ program that contains two user defined function Add() and Sum(). Access variables defined in the Function Add() in both Sum() and main() and report the behavior.

Lab Exercise # 2:

Write a C++ program that perform Basic Arithmetic operation using function sharing same global variables and report the ultimate advantage of using global variables.

Practical No. 18, String in C++

C++ Strings: Strings are used for storing text. A string variable contains a collection of characters surrounded by double quotes:

One of the most useful data types supplied in the C++ libraries is the string. A string is a variable that stores a sequence of letters or other characters, such as "Hello" or "May 10th is my birthday!". Just like the other data types, to create a string we first declare it, then we can store a value in it.

string testString;

testString = "This is a string.";

We can combine these two statements into one line:

string testString = "This is a string.";

Often, we use strings as output, and cout works exactly like one would expect:

cout << testString << endl;

will print the same result as

cout << "This is a string." << endl;

In order to use the string data type, the C++ string header <string> must be included at the top of the program. Also, you'll need to include using namespace std; to make the short name string visible instead of requiring the cumbersome std::string. Thus, you would have the following #include's in your program in order to use the string type

Counting the number of characters in a string

The length method returns the number of characters in a string, including spaces and punctuation. Like many of the string operations, length is a member function, and we invoke member functions using dot notation. The string that is the receiver is to the left of the dot, the member function we are invoking is to the right, (e.g. str.length()). In such an expression, we are requesting the length from the variable str.

```
#include <string>
#include <iostream>
using namespace std;
#include "console.h"
int main() {
string small, large;
small = "I am short";
large = "I, friend, am a long and elaborate string indeed";
```

```
cout << "The short string is " << small.length()
 << " characters." << endl;
cout << The long string is " << large.length()
 << " characters." << endl;
return 0;
}
```

Accessing individual characters:

Using square brackets, you can access individual characters within a string as if it's a char array. Positions within a string str are numbered from 0 through str.length() - 1. You can read and write to characters within a string using [].

```
#include <string>
#include <iostream>
using namespace std;
#include "console.h"
int main() {
string test;
test = "I am Q the omnipot3nt";
char ch = test[5]; // ch is 'Q'
test[18] = 'e'; // we correct misspelling of omnipotent
cout << test << endl;
cout << "ch = " << ch << endl;
return 0;
}
```

Appending to a string:

C++ strings are wondrous things. Suppose you have two strings, s1 and s2 and you want to create a new string of their concatenation. Conveniently, you can just write s1 + s2, and you'll get the result you'd expect. Similarly, if you want to append to the end of string, you can use the += operator. You can append either another string or a single character to the end of a string.

```
#include <string>
#include <iostream>
using namespace std;

int main() {
string firstname = "Ayesha";
string lastname = " Khan";
```

```
string fullname = firstname + lastname; // concat the two strings
fullname += ", Jr"; // append another string
 fullname += '.'; // append a single char
cout << firstname << lastname << endl;
cout << fullname << endl;
return 0;
}
```

The C-Style Character String

The C-style character string originated within the C language and continues to be supported within C++. This string is actually a one-dimensional array of characters which is terminated by a null character '\0'. Thus a null-terminated string contains the characters that comprise the string followed by a null.

The following declaration and initialization create a string consisting of the word "Hello". To hold the null character at the end of the array, the size of the character array containing the string is one more than the number of characters in the word "Hello."

char greeting[6] = {'H', 'e', 'l', 'l', 'o', '\0'};

```
#include <string>
#include <iostream>
using namespace std;

int main () {

   char greeting[6] = {'H', 'e', 'l', 'l', 'o', '\0'};

   cout << "Greeting message: ";
   cout << greeting << endl;

   return 0;
}

#include <string>
#include <iostream>
#include <cstring>

using namespace std;

int main () {

   char str1[10] = "Hello";
   char str2[10] = "World";
   char str3[10];
```

```
   int  len ;

   // copy str1 into str3
   strcpy( str3, str1);
   cout << "strcpy( str3, str1) : " << str3 << endl;

   // concatenates str1 and str2
   strcat( str1, str2);
   cout << "strcat( str1, str2): " << str1 << endl;

   // total lenghth of str1 after concatenation
   len = strlen(str1);
   cout << "strlen(str1) : " << len << endl;

   return 0;
}
```

Practical No. 18	Checked	Marks
Lab Exercise # 1:		
Lab Exercise # 2:		

Lab Exercise # 1:

Write a C++ program to convert a string “Pakistan Zindabad” to lowercase or uppercase.

Lab Exercise # 2:

Write a C++ program that to compare two strings s1= “Information Engineering” and s2 = “ information enginering” are equal or not.

Practical No. 19: File Handling in C++

File handling is used for store a data permanently in computer. Using file handling we can store our data in secondary memory (Hard disk).

How to achieve the File Handling

For achieving file handling we need to follow the following steps:-

STEP 1- Naming a file

STEP 2- Opening a file

STEP 3- Writing data into the file

STEP 4- Reading data from the file

STEP 5- Closing a file.

C++ Files

The fstream library allows us to work with files. To use the fstream library, include both the standard <iostream> AND the <fstream> header file:

Example

#include <iostream>

#include <fstream>

Create and Write To a File

To create a file, use either the ofstream or fstream class, and specify the name of the file. To write to the file, use the insertion operator (<<).

```
// basic file operations
#include <iostream>
#include <fstream>
using namespace std;

int main ()
 {
  ofstream myfile;
  myfile.open ("example.txt");
  myfile << "Writing this to a file.\n";
  myfile.close();
```

```
  return 0;
}
```

Alternatively

```
// writing on a text file
#include <iostream>
#include <fstream>
using namespace std;

int main ()
{
  ofstream myfile ("example.txt");
  if (myfile.is_open())
  {
    myfile << "This is a line.\n";
    myfile << "This is another line.\n";
    myfile.close();
  }
  else cout << "Unable to open file";
  return 0;
}
```

Read a File

To read from a file, use either the ifstream or fstream class, and the name of the file.

```
// reading a text file
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main () {
  string line;
  ifstream myfile ("example.txt");
  if (myfile.is_open())
  {
    while ( getline (myfile,line) )
    {
      cout << line << '\n';
    }
    myfile.close();
  }
```

```
  else cout << "Unable to open file";

  return 0;
}
```

```
#include <iostream>
#include <fstream>
using namespace std;

int main()

{
       fstream ob;
       ob.open("test.txt", ios::out);     //opening file in writing mode
       ob << "hello world\n";      //writing data to file
       ob << "this is my first file";
       ob.close();   //closing the file
       //again opening the file but in reading mode
       ob.open("test.txt", ios:: in);
       while (!ob.eof())
       {
              string str;
              //reading word by word from file and storing in str
              ob >> str;
              cout << str << "\n"; //printing str

       }
       ob.close();   //closing the file after use
       return 0;

}
```

Practical No. 19	Checked	Marks
Lab Exercise # 1:		
Lab Exercise # 2:		

Lab Exercise # 1:

Write a C++ program that create “ myfirstfile.txt” and save a paragraph about your department in the file.

Lab Exercise # 2:

Write a C++ program to confirm the above created “myfirstfile.txt” if didn’t exist the frogram generate an Error “The requested file not found”.

Practical No. 20: Errors in C++

Error is an illegal operation performed by the user which results in abnormal working of the program. Programming errors often remain undetected until the program is compiled or executed. Some of the errors inhibit the program from getting compiled or executed. Thus errors should be removed before compiling and executing.

In C++, we face different kinds of errors. These errors can be categorized into five different types.

1. **Syntax Error**
2. **Run-Time Error**
3. **Linker Error**
4. **Logical Error**
5. **Semantic Error**

Syntax errors: Errors that occur when you violate the rules of writing C/C++ syntax are known as syntax errors. This compiler error indicates something that must be fixed before the code can be compiled. All these errors are detected by compiler and thus are known as compile-time errors.

Most frequent syntax errors are:

1. Missing Parenthesis (})
2. Printing the value of variable without declaring it
3. Missing semicolon like this:

```
// C++ program to illustrate syntax error
#include <iostream>
using namespace std;

int main()
{
    int x = 10;
    int y = 15;

    cout << " "<< (x, y) // semicolon missed
    return 0;
}
```

Error:

error: expected ';' before '}' token

Syntax of a basic construct is written wrong. For example: while loop

```
// C++ program to illustrate syntax error
#include <iostream>
using namespace std;

int main(void)
{
    // while() cannot contain "." as an argument.
    while(.)
    {
        cout <<"hello";
    }
    return 0;
}
```

error: expected expression before '.' token

while(.)

In the given example, the syntax of while loop is incorrect. This causes a syntax error.

Run-time Errors:

Errors which occur during program execution(run-time) after successful compilation are called run-time errors. One of the most common run-time error is division by zero also known as Division error. These types of error are hard to find as the compiler doesn't point to the line at which the error occurs.

```
// C++ program to illustrate run-time error
#include <iostream>
using namespace std;

int main()
{
    int n = 9, div = 0;

    // wrong logic
    // number is divided by 0,
    // so this program abnormally terminates
    div = n/0;

   cout << "result = "<< div;
   return 0;
}
```

Linker Errors:

These error occurs when after compilation we link the different object files with main's object using Ctrl+F9 key(RUN). These are errors generated when the executable of the program cannot be generated. This may be due to wrong function prototyping, incorrect header files. One of the most common linker error is writing Main() instead of main().

```
// C++ program to illustrate linker error
#include <iostream>
using namespace std;

int Main() // Here Main() should be main()
{
    int a = 10;
    cout << " "<< a;
    return 0;
}
```

Logical Errors:

On compilation and execution of a program, desired output is not obtained when certain input values are given. These types of errors which provide incorrect output but appear to be error free are called logical errors. These are one of the most common errors done by beginners of programming.

These errors solely depend on the logical thinking of the programmer and are easy to detect if we follow the line of execution and determine why the program takes that path of execution.

```
// C++ program to illustrate logical error
#include <iostream>
using namespace std;
int main()
{
    int i = 0;

    // logical error : a semicolon after loop
    for(i = 0; i < 3; i++);
    {
       cout << "loop ";
        continue;
    }
    return 0;
}
```

Semantic errors :

This error occurs when the statements written in the program are not meaningful to the compiler.

```
// C++ program to illustrate semantic error
#include<iostream>
Using namespace std;
int main()
{
    int a, b, c;
    a + b = c; //semantic error
}
```

Practical No. 20	Checked	Marks
Lab Exercise # 1:		
Lab Exercise # 2:		
Lab Exercise # 3:		

Lab Exercise # 1:

Find Error in the code and define the type of error.

```
// identify and fix the error
#include<iostream>
Using namespace std;
Int main
{
int sum,sub,mult,div,module;
int a,b;
cout<<"Enter value of a ="
cin<<a;
cout<<"Enter value of b =;
cin<<b;
sum=a+b;
sub=a-bmult=a*b;
div=a/'b';
module=a%b;
```

Lab Exercise # 2:

Identify and Fix the error.

```
// identify and fix the error
#include<iostream>
int main()
{
     int i=10, j=5;
     int modResult=0;
     int divResult=0;
     modResult = i%j;
     cout<<modResult<<" ";
     divResult = i/modResult;
     cout<<divResult;
     return 0;
}
```

Lab Exercise # 3:

Identify and Fix the error.

```
#include<iostream>
using namespace std;
int fun()
{
    return 1;
}
float fun1()
{
    return 10.23;
}

int main()
{
     cout<<fun()<<"  ";
     cout<<fun1()<<"  ";
     return 0;
}
```